Experiment 1: Vector and Matrix Operations for Deep Learning

1. AIM

To perform vector and matrix operations using NumPy, calculate cosine similarity between customers, implement a dense neural network layer manually, and visualize customer–book recommendation scores using a heatmap with Matplotlib.

2. MATHEMATICAL FORMULAE

1. Dot Product

The dot product of two vectors A and B is:

$$A \cdot B = \sum_{i=1}^{n} A_i B_i$$

2. Matrix Multiplication

For matrices A and B:

$$C = AB$$

Each element is calculated as:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

3. Transpose of a Matrix

The transpose interchanges rows and columns.

$$(A^T)_{ij} = A_{ji}$$

4. Inverse of a Matrix

For a square, non-singular matrix A:

$$AA^{-1} = A^{-1}A = I$$

The inverse exists only when:

$$\det(A) \neq 0$$

5. Element-wise Operations

Addition:

$$C_{ij} = A_{ij} + B_{ij}$$

Multiplication:

$$C_{ij} = A_{ij} \times B_{ij}$$

6. Cosine Similarity

Cosine similarity measures the similarity between two customer rating vectors.

$$\boxed{\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}}$$

where:

$$\|A\| = \sqrt{\sum_{i=1}^{n} A_i^2}$$

The similarity value ranges from -1 to 1. A value closer to 1 indicates greater similarity.

7. Dense Neural Network Layer

The output of a dense layer is:

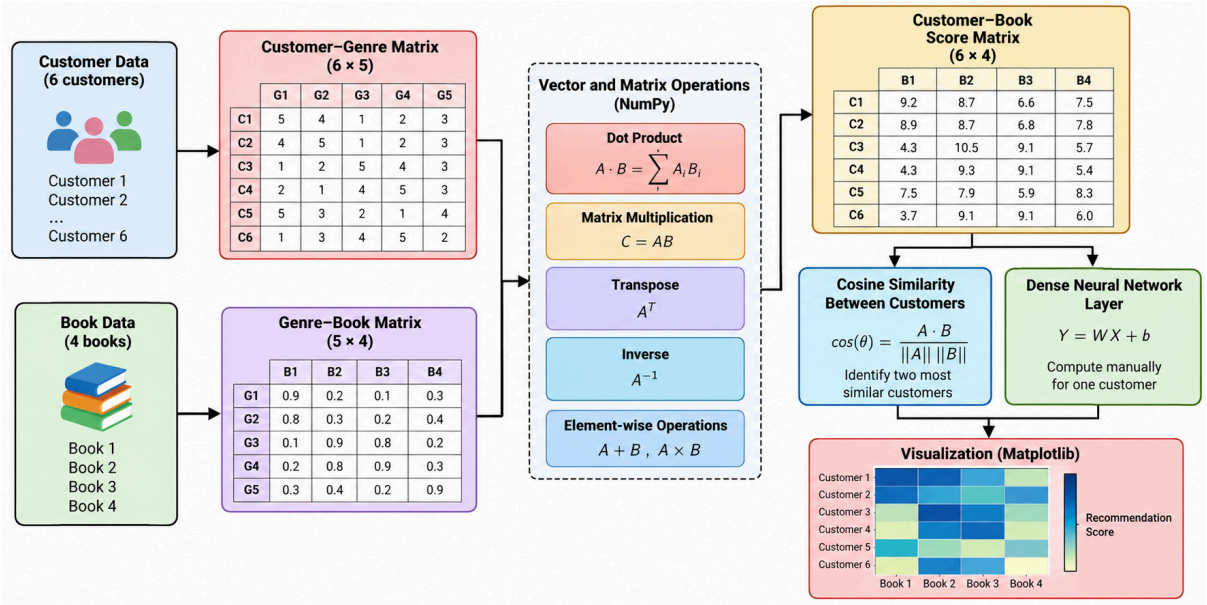
$$\boxed{Y = WX + b}$$

where:

- X = Input vector
- W = Weight matrix
- b = Bias vector
- Y = Output vector

For a layer with an activation function:

$$Y = f(WX + b)$$



CODE

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Customer-Genre Matrix (6 customers, 5 genres)
customer_genre = np.array([
    [5, 4, 1, 2, 3],
    [4, 5, 1, 2, 3],
    [1, 2, 5, 4, 3],
    [2, 1, 4, 5, 3],
    [5, 3, 2, 1, 4],
    [1, 3, 4, 5, 2]
])

# 2. Genre-Book Matrix (5 genres, 4 books)
genre_book = np.array([
    [0.9, 0.2, 0.1, 0.3],
    [0.8, 0.3, 0.2, 0.4],
    [0.1, 0.9, 0.8, 0.2],
    [0.2, 0.8, 0.9, 0.3],
    [0.3, 0.4, 0.2, 0.9]
])

print("Customer-Genre Matrix:\n", customer_genre)
print("\nGenre-Book Matrix:\n", genre_book)

# 3. Dot Product
print("\nDot Product of first two customers:")
print(np.dot(customer_genre[0], customer_genre[1]))

# 4. Matrix Multiplication
score_matrix = np.dot(customer_genre, genre_book)
print("\nCustomer-Book Score Matrix:\n", score_matrix)

# 5. Transpose
print("\nTranspose of Customer-Genre Matrix:\n",
      customer_genre.T)

# 6. Inverse of a square matrix
square_matrix = np.array([
```

```
    [2, 1],
    [1, 2]
])

print("\nInverse of Square Matrix:\n",
      np.linalg.inv(square_matrix))

# 7. Element-wise Operations
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("\nElement-wise Addition:\n", A + B)
print("\nElement-wise Multiplication:\n", A * B)

# 8. Cosine Similarity
def cosine_similarity(a, b):
    denominator = np.linalg.norm(a) * np.linalg.norm(b)

    if denominator == 0:
        return 0

    return np.dot(a, b) / denominator

similarity_matrix = np.zeros((6, 6))

for i in range(6):
    for j in range(6):
        similarity_matrix[i, j] = cosine_similarity(
            customer_genre[i],
            customer_genre[j]
        )

print("\nCustomer Cosine Similarity Matrix:\n",
      np.round(similarity_matrix, 3))

# Find the two most similar distinct customers
upper_triangle = np.triu(similarity_matrix, k=1)
i, j = np.unravel_index(
    np.argmax(upper_triangle),
    upper_triangle.shape
)

print(f"\nMost Similar Customers: {i+1} and {j+1}")
print("Cosine Similarity:", round(similarity_matrix[i, j], 3))

# 9. Dense Neural Network Layer: Y = WX + b
X = customer_genre[0]

W = np.array([
    [0.2, 0.4, 0.1, 0.3, 0.5],
    [0.3, 0.1, 0.4, 0.2, 0.6]
])

b = np.array([0.1, 0.2])

Y = np.dot(W, X) + b

print("\nDense Layer Output:")
print("X =", X)
print("Y = WX + b =", Y)
```

```
# 10. Heatmap of Customer-Book Scores
plt.figure(figsize=(8, 5))

plt.imshow(score_matrix, cmap="YlGnBu", aspect="auto")
plt.colorbar(label="Recommendation Score")

plt.xticks(
    range(4),
    ["Book 1", "Book 2", "Book 3", "Book 4"]
)

plt.yticks(
    range(6),
    ["Customer 1", "Customer 2", "Customer 3",
     "Customer 4", "Customer 5", "Customer 6"]
)

plt.title("Customer-Book Recommendation Scores")
plt.xlabel("Books")
plt.ylabel("Customers")

for i in range(6):
    for j in range(4):
        plt.text(
            j, i, f"{score_matrix[i, j]:.2f}",
            ha="center", va="center"
        )

plt.tight_layout()
plt.show()
```

OUTPUT

```
Customer-Genre Matrix:
 [[5 4 1 2 3]
 [4 5 1 2 3]
 [1 2 5 4 3]
 [2 1 4 5 3]
 [5 3 2 1 4]
 [1 3 4 5 2]]

Genre-Book Matrix:
 [[0.9 0.2 0.1 0.3]
 [0.8 0.3 0.2 0.4]
 [0.1 0.9 0.8 0.2]
 [0.2 0.8 0.9 0.3]
 [0.3 0.4 0.2 0.9]]

Dot Product of first two customers:
54

Customer-Book Score Matrix:
 [[9.1 5.9 4.5 6.6]
 [9.  6.  4.6 6.7]
 [4.7 9.7 8.7 6. ]
 [4.9 9.5 8.7 6. ]
 [8.5 6.1 4.4 7. ]
 [5.3 9.5 8.8 5.6]]

Transpose of Customer-Genre Matrix:
 [[5 4 1 2 5 1]
 [4 5 2 1 3 3]
 [1 1 5 4 2 4]
 [2 2 4 5 1 5]
 [3 3 3 3 4 2]]

Inverse of Square Matrix:
 [[ 0.66666667 -0.33333333]
 [-0.33333333  0.66666667]]

Element-wise Addition:
 [[ 6  8]
 [10 12]]

Element-wise Multiplication:
 [[ 5 12]
 [21 32]]

Customer Cosine Similarity Matrix:
 [[1.    0.982 0.636 0.673 0.964 0.673]
 [0.982 1.    0.655 0.655 0.927 0.709]
 [0.636 0.655 1.    0.964 0.673 0.964]
 [0.673 0.655 0.964 1.    0.691 0.945]
 [0.964 0.927 0.673 0.691 1.    0.636]
 [0.673 0.709 0.964 0.945 0.636 1.   ]]

Most Similar Customers: 1 and 2
Cosine Similarity: 0.982

Dense Layer Output:
X = [5 4 1 2 3]
Y = WX + b = [4.9 4.7]
```


Customer-Book Recommendation Scores
Customer 1
Customer 2
Customer 3
Customer 4
Customer 5
Customer 6
Customers
9.10
5.90
4.50
6.60
9.00
6.00
4.60
6.70
4.70
9.70
8.70
6.00
4.90
9.50
8.70
6.00
8.50
6.10
4.40
7.00
5.30
9.50
8.80
5.60
Book 1
Book 2
Book 3
Book 4
Books
Recommendation Score
9
8
7
6
5